Java: Interface in Depth

"Concept && Coding" YT Video Notes

- What is Interface and How to define
- Why we need interface
- Methods in interface
- Fields in an interface
- Interface Implementation
- Nested Interface
- Difference between Interface and Abstract class.

Java8 Interface features:

- Default method
- Static method
- Functional interface and Lambda expression

Java9 Interface feature:

- Private method and
- Private Static method

Report Abuse

What is Interface?

Interface is something which helps 2 system to interact with each other, without one system has to know the details of other.

Or in simple term i can say, it helps to achieve ABSTRACTION.

How to define the Interface?

Interface declaration consist of

- Access specifier
- "Interface" keyword
- Interface Name
- Comma separated list of parent interfaces
- Body

Only **Public** and **Default** Modifiers are allowed. **Protected** and **private** are not allowed.

```
public interface Bird {

    no usages
    public void fly();
}
```

```
interface Bird {

    no usages
    public void fly();
}
```

Comma separated list of parent Interfaces (it can extend from **Class**) Example:

```
public interface NonFlyingBirds extends Bird, LivingThings{

    no usages
    public void canRun();
}
```

Why we need interface?

1. Abstraction:

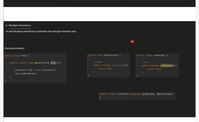
Using interface, we can achieve full Abstraction means, we can define WHAT class must do, but not HOW it will do

```
public interface Bird {

    no usages
    public void fly();
}
```

```
public class Eagle implements Bird {

    1 usage
    @override
    public void fly() {
        //the internal process of flying take place here
    }
}
```


2. Multiple Inheritance:
2. Java Multiple inheritance is possible only through interface only.
Interface problem:
interface Test {
public void test();
}
interface Test {
public void test();
}
class Test implements Test {
@Override
public void test() {
System.out.println("Hello");
}
}
interface Test {
int a = 10;
}
interface Test {
int a = 20;
}
class Test implements Test {
public void test() {
System.out.println(a);
}
}
interface Test {
int a = 10;
}
interface Test {
int a = 20;
}
class Test implements Test {
public void test() {
System.out.println(a);
}
}
interface Test {
void test();
}
class Test {
public void test() {
System.out.println("Hello");
}
}
interface Test {
void test();
}
class Test {
public void test() {
System.out.println("Hello");
}
}

How to extend interface, that contains Default Method:

1st Way:

```java
public interface LivingThing {

    default boolean canBreathe(){
        return true;
    }
}
```

```java
public interface Bird extends LivingThing{

}
```

```java
public class Eagle implements Bird{

}
```

```java
public class Main {

    public static void main(String args[]){

        Eagle eagleObj = new Eagle();
        eagleObj.canBreathe();

    }
}
```

2nd Way:

```java
public interface LivingThing {

    default boolean canBreathe(){
        return true;
    }
}
```

```java
public interface Bird extends LivingThing{

    boolean canBreathe();
}
```

```java
public class Eagle implements Bird{

}
```

```java
public class Eagle implements Bird{

    @Override
    public boolean canBreathe() {
        return true;
    }
}
```

3rd Way:

```java
public interface LivingThing {

    default boolean canBreathe(){
        return true;
    }
}
```

```java
public interface Bird extends LivingThing {

    default boolean canBreathe() {
        boolean canBreatheOrNot = LivingThing.super.canBreathe();
        // do something else
        return canBreatheOrNot;
    }
}
```

```java
public class Eagle implements Bird{
}
```

2. Static Method (Java8):

- We can provide the implementation of the method in interface.
- But it can not be overridden by classes which implement the interface.
- We can access it using Interface name itself.
- Its by default public.

```java
public interface Bird {

    static boolean canBreathe() {
        return true;
    }
}
```

```java
public class Eagle implements Bird{

    public void digestiveSystemTestMethod() {

        if(Bird.canBreathe()){

            //do something
        }
    }
}
```

If you try to override it,
it will just be treated as new method in Eagle class

```java
public class Eagle implements Bird{

    public boolean canBreathe() {
        System.out.println("in interface");
        return true;
    }
}
```

If you try to add @override annotation,
it will throw compilation error

```java
public class Eagle implements Bird{

    @Override
    public boolean canBreathe() {
        System.out.println("in interface");
        return true;
    }
}
```

3. Private Method and Private Static method (Java9):

- We can provide the implementation of method but as a private access modifier in interface.
- It brings more readability of the code. For example if multiple default method share some code, that this can help.
- It can be defined as static and non-static.
- From Static method, we can call only private static interface method.
- Private static method, can be called from both static and non static method.
- Private interface method can not be abstract. Means we have to provide the definition.
- It can be used inside of the particular interface only.

```java
public interface Bird {

    void canFly();//this is equivalent to public abstract void canFly();
    void canFly();//this is equivalent to public abstract void canFly();

    public default void minimumFlyingHeight() {
        myStaticPublicMethod();  //calling static method
        myPrivateMethod(); //calling private method
        myPrivateStaticMethod();   //calling private static method
    }

    static void myStaticPublicMethod() {
        myPrivateStaticMethod(); //from static we can call other static method only
    }

    private void myPrivateMethod(){
        // private method implementation
    }

    private static void myPrivateStaticMethod(){
        // private static method implementation
    }
}
```